# Product Label Analyzer

Revised architecture and implementation roadmap ? MVP-first, evidence-grounded, and auditable

## Verdict

The original proposal has the right product principle?do not produce a black-box ?healthy/unhealthy? score?but it is too broad for an efficient first build. Start with one product category and one jurisdiction, establish extraction and evidence-quality metrics, and add jurisdictions and advanced retrieval only after the core workflow is reliable.

## What should change immediately

? **Reduce scope:** choose one category (for example packaged foods) and one jurisdiction (India or the market you will operate in).

? **Separate certainty levels:** deterministic label checks and regulatory rules must not be blended with LLM-generated interpretation.

? **Delay infrastructure:** use SQLite/FTS5 and a small curated corpus before PostgreSQL, vector search, rerankers, and a distributed job system.

? **Make evidence first-class:** retain image regions, OCR confidence, source version, retrieval date, quoted text, and the exact rule used.

? **Measure quality:** create a labelled evaluation set before optimizing models or prompts.

## Target outcome

A user uploads label images and receives a claim-by-claim report with extracted text, confidence, applicable rules, supporting evidence, limitations, and an explicit **Unable to verify** state. The system should support audit and correction?not pretend to replace legal, regulatory, or medical advice.

## Recommended MVP boundary

| Include in MVP | Defer until validated |
|---|---|
| Image quality checks, OCR, structured extraction, ingredient normalization, one-jurisdiction deterministic checks, curated evidence search, citations, human-review flag, report export. | Three-jurisdiction parity, broad literature search, autonomous regulatory ingestion, health scoring, multi-agent orchestration, fine-tuned models, vector DB and cross-encoder reranking. |

# 1. Recommended System Architecture

Use a staged pipeline with typed contracts. Each stage stores its input, output, confidence, and provenance so failures are visible and reproducible.

| Stage | MVP implementation | Why this is efficient |
|---|---|---|
| 1. Intake | FastAPI upload; validate MIME, size, dimensions; encrypt or isolate files; retain original. | Prevents unusable inputs and supports audit. |
| 2. Image quality | OpenCV/Pillow: orientation, blur, glare, crop, perspective; return quality score and user guidance. | Improves OCR more cheaply than adding a larger model. |
| 3. OCR | One primary OCR engine; preserve word boxes and confidence. Vision model only for low-confidence regions. | Avoids paying for vision inference on every image. |
| 4. Parsing | Pydantic schema plus constrained LLM extraction from OCR; validate required fields and retain raw text. | Typed output is testable and correctable. |
| 5. Normalization | Versioned alias/code tables for ingredients, units, allergens, and claims. | Deterministic and explainable; no embedding required initially. |
| 6. Evidence | Curated official corpus + keyword/FTS search; claim/rule matching; citations with quotes. | Higher precision and traceability than open-ended web RAG. |
| 7. Assessment | Rule engine for checks; LLM only summarizes supplied evidence into a fixed output schema. | Prevents unsupported regulatory conclusions. |
| 8. Report | JSON API first; HTML/PDF rendering second; include uncertainty and review flags. | Keeps the core product machine-readable and testable. |

### Control flow

**Upload ? Validate ? Preprocess ? OCR ? Parse ? Normalize ? Deterministic checks + evidence search (parallel) ? Sufficiency gate ? Assessment ? Citation validation ? Report**

Important: the sufficiency gate must be able to stop synthesis. If required source text, image evidence, or confidence thresholds are missing, return **Unable to verify** rather than asking the LLM to fill the gap.

# 2. Data and Evidence Contracts

The most important architectural improvement is to make provenance and versioning explicit. Store the evidence that led to each conclusion, not just the conclusion.

## Minimum entities

| Entity | Required fields |
|---|---|
| Product / Analysis | analysis_id, product metadata, jurisdiction, pipeline version, created_at, status, reviewer status. |
| Image / OCR span | image hash, page/side, crop coordinates, OCR text, word boxes, confidence, preprocessing version. |
| Field / Ingredient / Claim | raw value, canonical value, source span, normalization method/version, confidence, warnings. |
| Source document | publisher, jurisdiction, URL, document identifier, effective date, retrieval date, content hash, source tier. |
| Evidence item | source_id, quoted passage, location/page/section, query, rank, relevance, evidence quality, retrieved_at. |
| Finding | subject, rule/claim, result, rationale, evidence_ids, confidence, limitations, review_required. |

## Source hierarchy

? **Tier 1:** current official regulation, standard, guidance, or regulator database.

? **Tier 2:** official regulator assessment or authoritative public-health guidance.

? **Tier 3:** peer-reviewed systematic reviews and primary studies.

? **Tier 4:** secondary summaries. Use for discovery, not as the sole basis of a compliance finding.

Do not silently overwrite source content. Ingest immutable snapshots or hashes, record effective/retrieval dates, and mark superseded versions. For India, preserve the exact Gazette/notification or official version when a compendium says the original prevails.

## 3. Keep Rules Deterministic; Keep the LLM Constrained

A safer and more efficient division of labour is:

| Component | Allowed responsibility | Not allowed |
|---|---|---|
| OCR / vision | Read text and layout; identify regions needing review. | Decide legal compliance. |
| Parser LLM | Map OCR into a strict schema; cite source spans; report uncertainty. | Invent missing ingredients, quantities, or claims. |
| Rule engine | Apply versioned thresholds, conditions, units, and jurisdiction rules. | Use semantic similarity as proof of compliance. |
| Evidence retrieval | Find candidate passages from a curated corpus and rank them. | Treat search snippets or unverified web pages as authority. |
| Synthesis LLM | Turn supplied findings into readable prose and structured rationale. | Add new facts, rules, citations, or confidence not present in inputs. |

### Fixed result vocabulary

? PASS ? supplied evidence and rule conditions are satisfied.

? FAIL ? supplied evidence contradicts a rule or claim requirement.

? REVIEW ? evidence is incomplete, ambiguous, or requires human/legal interpretation.

? UNABLE_TO_VERIFY ? required evidence is missing or below the sufficiency threshold.

? NOT_APPLICABLE ? the rule or claim does not apply to the selected jurisdiction/category.

Avoid a single ?health score.? Report three independent dimensions: **label extraction confidence**, **regulatory status**, and **scientific evidence strength**. A product can be compliant while a health claim remains scientifically uncertain, or vice versa.

# 4. Technology Choices for the Efficient Path

| Area | Start with | Upgrade trigger |
|---|---|---|
| API / schema | FastAPI + Pydantic. | Keep; add async job API when analyses exceed request timeouts. |
| Storage | SQLite for local MVP; object storage for images in production. | Move to PostgreSQL when concurrent users, audit queries, or multi-tenant needs justify it. |
| Search | SQLite FTS5/BM25 over a curated corpus. | Add pgvector/Qdrant only after lexical recall is measured and insufficient. |
| OCR | One tested OCR engine with boxes/confidence. | Add vision fallback for measured low-confidence layouts. |
| LLM | One model, structured output, low temperature, logged prompt/schema version. | Model routing or fine-tuning only after an evaluation set shows a gap. |
| Frontend | Simple React/Vite or Streamlit prototype. | React workflow once review, corrections, and history are validated. |
| Jobs / caching | Explicit analysis state; local background worker for MVP; cache by image/source/model hash. | Celery/RQ/queue when throughput or retries require it. |

## Security and privacy are not optional add-ons

? Treat uploaded images as sensitive business data; define retention and deletion controls.

? Validate file type by content, limit size/pixels, strip unsafe metadata, and isolate processing.

? Keep API keys server-side; redact secrets and personal data from logs.

? Do not send images or label data to a third-party model without an explicit data-processing decision.

? Add authentication, authorization, rate limits, and audit logs before exposing the service publicly.

# 5. Evaluation-Driven Roadmap

Build the evaluation set before adding sophisticated retrieval. It is the fastest way to know whether a change improves the product.

| Phase | Deliverable | Exit criteria |
|---|---|---|
| 0. Dataset | 50?100 representative products, front/back images, manually corrected text, ingredients, claims, and expected findings. | Ground truth and reviewer guidelines exist. |
| 1. Extraction | Upload, quality checks, OCR, schema parser, correction UI/API. | Measure field accuracy, OCR confidence calibration, and missing-field rate. |
| 2. Normalization | Versioned ingredient/additive/allergen/claim dictionaries. | Measure canonicalization precision/recall and unknown-term rate. |
| 3. One jurisdiction | Curated official corpus and deterministic checks for one category. | Every finding has a source quote and rule version; legal review of examples. |
| 4. Evidence and report | FTS retrieval, sufficiency gate, claim-by-claim report, HTML/PDF export. | Citation precision and unsupported-claim rate meet agreed thresholds. |
| 5. Expansion | Scientific literature, second/third jurisdictions, vector retrieval only if needed. | Each addition has its own corpus, rule mapping, tests, and evaluation results. |

## Test categories

? OCR: rotated text, glare, curved packs, multilingual labels, tiny nutrition tables.

? Parsing: missing back image, duplicate ingredients, ambiguous units, claims split across lines.

? Rules: jurisdiction differences, effective dates, exemptions, unit conversion, boundary values.

? Evidence: stale sources, conflicting documents, irrelevant retrieval, no supporting passage.

? Safety: oversized files, malicious uploads, prompt injection in label text, secret leakage in logs.

## 6. Suggested Repository Shape

Keep the repository organized around contracts and pipeline stages rather than prematurely splitting every technology into its own subsystem.

```
product-label-analyzer/
app/ api/ # upload, analysis, reports domain/ # typed schemas, result vocabulary pipeline/ # quality, OCR, parsing, normalization rules/ # versioned deterministic checks evidence/ # source ingestion, FTS, citation resolver services/ # orchestration, storage, model adapters frontend/ # upload, review, report data/ fixtures/ # small test corpus only sources/ # immutable metadata/manifests scripts/ # source ingestion and evaluation tests/ unit/ integration/ evaluation/ README.md ARCHITECTURE.md .env.example docker-compose.yml
```

Avoid committing real product images or downloaded regulatory corpora if licensing, privacy, or size is unclear. Store manifests and reproducible ingestion scripts instead.

## 7. Final Recommendation

Proceed with the project, but do not implement the original 12-phase stack in sequence. The efficient order is: **dataset ? extraction quality ? one-jurisdiction rules ? provenance/citations ? evaluation ? expansion**. This reduces cost and risk, makes failures diagnosable, and creates a defensible foundation for later RAG and multi-jurisdiction support.

This architecture is a software design recommendation, not legal, regulatory, medical, or compliance advice. Validate rule interpretations with qualified domain experts before relying on production findings.